Quadruped Locomotion with Central Pattern Generators and Deep Reinforcement Learning

Amine Tourki*[1] and Albian Salihu†[1]

[1]Department of Robotics, EPFL

January 6, 2023

Abstract

In this article, we present the implementation of two different approaches for locomotion on a quadruped robot: Central Pattern Generators (CPG) and Reinforcement Learning (RL). We will cover the specific implementation of each method and present the main results in terms of control and overall performance.

1 Introduction

The use of neural circuits has been a popular approach for the task of locomotion in quadruped robots. Two very different methods in essence have been particularly used in the last years: Bio-inspired circuits in contrast to Deep Neural Networks. In this article we will firstly implement a self organizing biological neural circuits, namely Central Pattern Generators (CPG) with different gaits. In the second part, we will implement a Deep Reinforcement Algorithm using a Markov Decision Process. Both methods will then be evaluated and tested for optimal performances.

2 Central Pattern Generator

In this section, we present the first used approach for locomotion which relies on Central Pattern Generators

2.1 Methods

We will present the implementation[1] of the locomotion controllers based on Central Pattern Generators, in particular the main equations used for the design of the CPG.
The CPG is modeled with coupled oscillators. Each *oscillator* i is of the form:

$$\dot{r}_i = -\alpha(\mu - r_i^2)r_i \tag{1}$$

*amine.tourki@epfl.ch
†albian.salihu@epfl.ch

$$\dot{\theta}_i = \omega_i + \sum_{j=0}^{3} r_j \omega_{ij} sin(\theta_j - \theta_i - \phi_{ij}) \tag{2}$$

where r_i is the current amplitude of the oscillator and θ_i is the phase of the oscillator. α is a positive constant that controls the speed of convergence to the limit cycle, ω_i is the natural frequency of oscillations, w_{ij} is the coupling strength between oscillators i and j, and ϕ_{ij} is the desired phase offset between oscillators i and j.

The natural frequency ω_i is split between ω_{swing} and ω_{stance}, where the phase variable for oscillator i will dictate the stance/swing status of leg i. We define the swing phase as $0 \leq \theta_i \leq \pi$, and stance phase as $\pi < \theta_i \leq 2\pi$.

To evaluate the gait, we define the duty cycle as the time it takes to go through one full cycle of stance + swing phases. The duty ratio D is the duration of one cycle in which the foot is in stance phase T_{stance} over the duration of one cycle in which the foot is in swing phase T_{swing}, or $D = T_{stance}/T_{swing}$

The output of the CPG is then mapped from its states to the Cartesian foot positions in the leg xz plane. This is accomplished using inverse kinematics with:

$$x_{foot} = -d_{step} r_i cos(\theta_i) \tag{3}$$

$$z_{foot} = \begin{cases} -h + g_c sin(\theta_i), & \text{if } sin(\theta_i) > 0 \\ -h + g_p sin(\theta_i), & \text{otherwise} \end{cases} \tag{4}$$

where d_{step} is the step length, h is the robot height, g_c is the max ground clearance during swing, and g_p is the max ground penetration during stance.

For the control, a joint PD control and Cartesian PD control are implemented. For each *leg i* of the robot we compute the torque using this equation:

$$\tau_{joint} = K_{p,joint}(q_d - q) + K_{d,joint}(\dot{q}_d - \dot{q}) \tag{5}$$

where q_d are the desired joint angles, $\dot{q}_d$ are the desired joint velocities, and q and $\dot{q}$ are the current joint angles and joint velocities, respectively. $K_{p,joint}$ and $K_{d,joint}$ are vectors of proportional and derivative gains.
To improve tracking performance, Cartesian PD controllers can be added. Given the current foot position and foot velocity, the corresponding motor torques to track the desired quantities can be computed with:

$$\tau_{cartesian} = J^T(q)[K_{p,cartesian}(p_d - p) + K_{d,cartesian}(v_d - v)] \tag{6}$$

where $J(q)$ is the foot Jacobian at joint configuration q, and $K_{p,cartesian}$ and $K_{d,cartesian}$ are diagonal matrices of proportional and derivative gains.
The final control is the sum of the joint PD control and Cartesian PD control:

$$\tau_{total} = \tau_{joint} + \tau_{cartesian} \tag{7}$$

2.2 Results

From the desired footfall sequences in fig1-4, the coupling matrices in equation 2 are designed as follows:

$$\phi_{trot}= \begin{bmatrix} 0 & -\pi & -\pi & 0 \\ \pi & 0 & 0 & \pi \\ \pi & 0 & 0 & \pi \\ 0 & -\pi & -\pi & 0 \end{bmatrix}$$

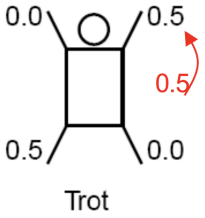

Figure 1: Trot gait: coupling matrix (left), footfall sequence (right)

$$\phi_{walk}= \begin{bmatrix} 0 & -\pi & -\frac{\pi}{2} & \frac{\pi}{2} \\ \pi & 0 & \frac{\pi}{2} & \frac{3\pi}{2} \\ \frac{\pi}{2} & -\frac{\pi}{2} & 0 & \pi \\ -\frac{\pi}{2} & -\frac{3\pi}{2} & -\pi & 0 \end{bmatrix}$$

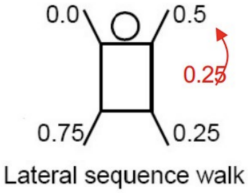

Figure 2: Lateral sequence walk gait gait: coupling matrix (left), footfall sequence (right)

$$\phi_{bound}= \begin{bmatrix} 0 & 0 & -\pi & -\pi \\ 0 & 0 & -\pi & -\pi \\ \pi & \pi & 0 & 0 \\ \pi & \pi & 0 & 0 \end{bmatrix}$$

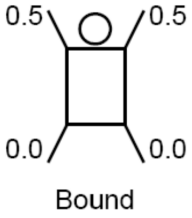

Figure 3: Bound gait: coupling matrix (left), footfall sequence (right)

$$\phi_{pace}= \begin{bmatrix} 0 & -\pi & 0 & -\pi \\ \pi & 0 & \pi & 0 \\ 0 & -\pi & 0 & -\pi \\ \pi & 0 & \pi & 0 \end{bmatrix}$$

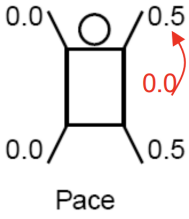

Figure 4: Pace gait: coupling matrix (left), footfall sequence (right)

The coupling matrices described in figure 1-4 produce the CPG states $(r, \theta, \dot{r}, \dot{\theta})$ in figure 5 with the initial swing and stance frequencies ($\omega_{swing} = 10\pi, \omega_{stance} = 4\pi$). The amplitude r of the CPG converges to 1 and can be better seen in figure 6. Both Joint and Cartesian PD are used in this case.

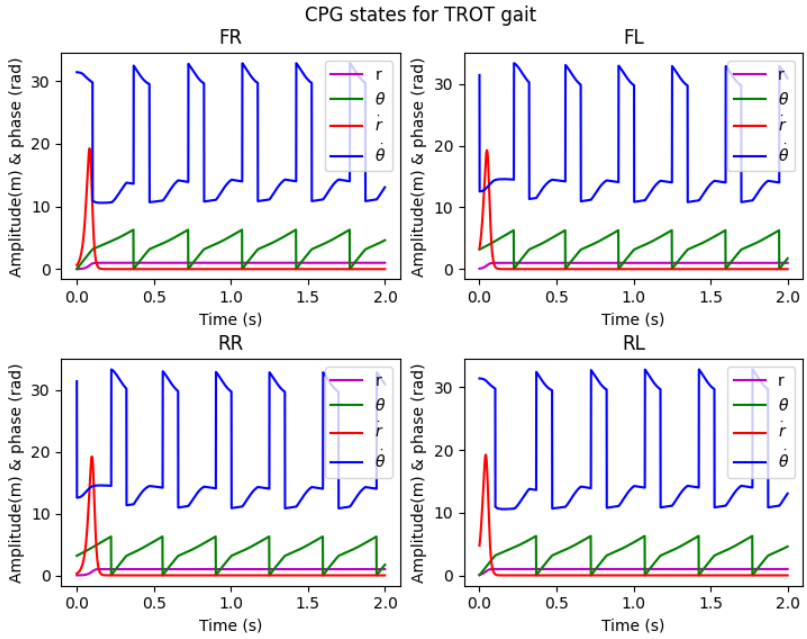

Figure 5: CPG states $(r, \theta, \dot{r}, \dot{\theta})$ of the four legs of the quadruped robot for a sequence of 2 seconds

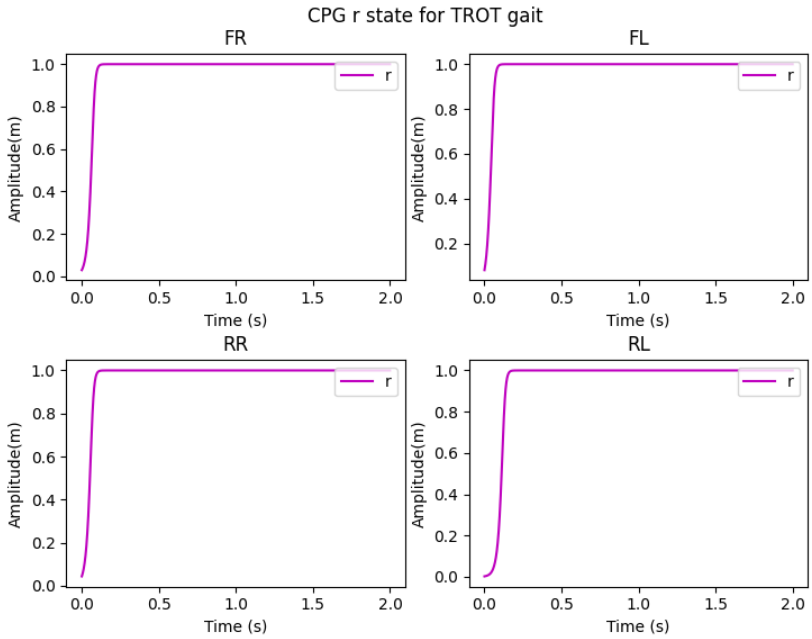

Figure 6: CPG amplitude of the four legs of the quadruped robot for a sequence of 2 seconds

In figures 7-9 we plot a comparison between the actual foot position and the desired foot position for all axes in a Trot gait (the other gates have similar results) with default gains ($\mathbf{K_p, joint} = 100$, $\mathbf{K_d, joint} = 2$, $\mathbf{K_p, cartesian} = 500$, $\mathbf{K_d, cartesian} = 20$). It is to be noted that the Robot is unstable and falls to the ground when a simple Cartesian PD only is used. Tuning of the gains is needed in this case. Results of the tuned gains can be seen in figures 10-12. The tuned values (Table 1) are discussed in section 2.3.1

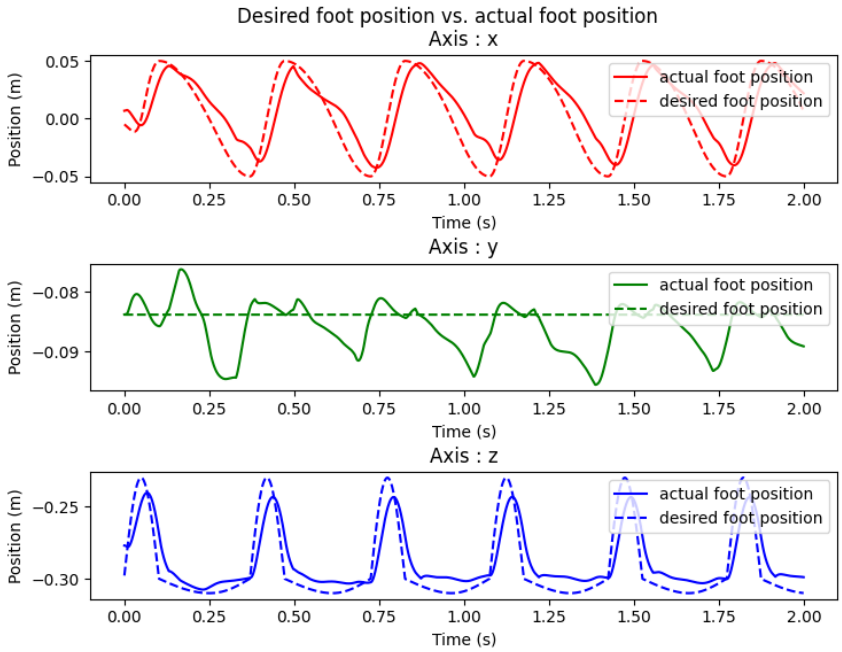

Figure 7: Desired vs actual foot position with both Joint and Cartesian PD for a sequence of 2 seconds (default gains)

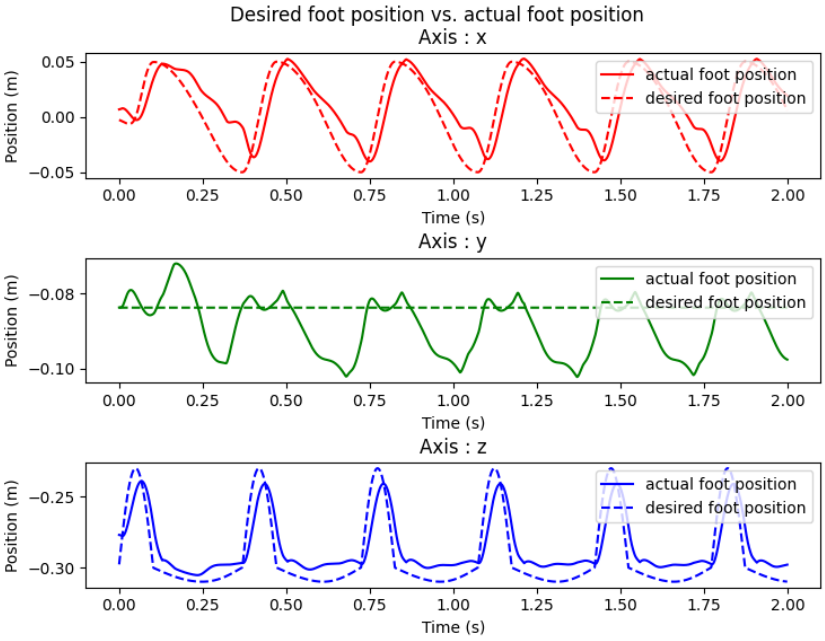

Figure 8: Desired vs actual foot position with only a Joint PD control for a sequence of 2 seconds (default gains)

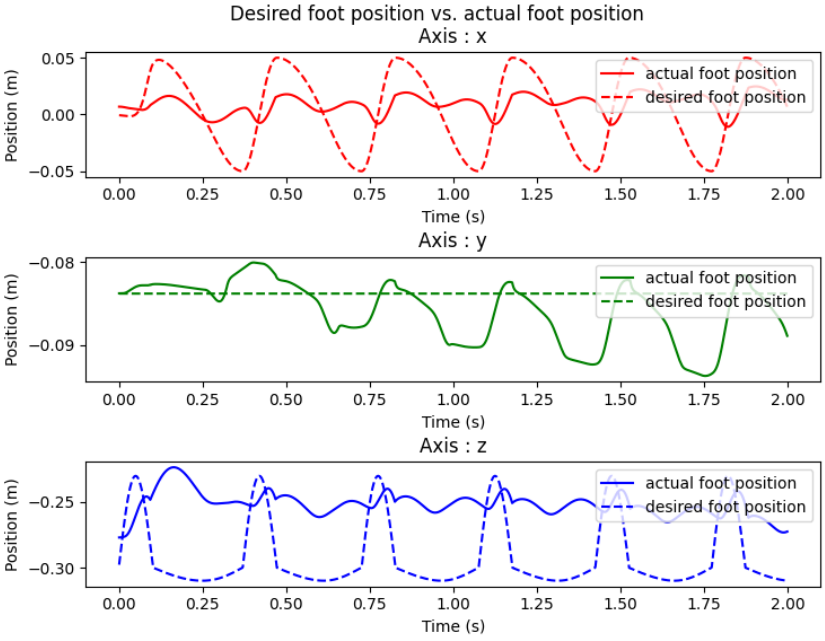

Figure 9: Desired vs actual foot position with only a Cartesian PD control for a sequence of 2 seconds (default gains)

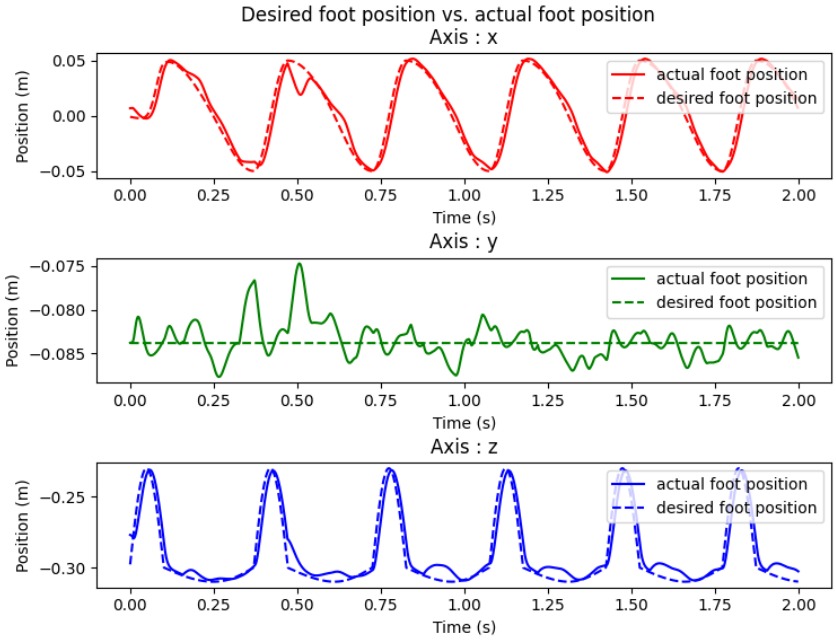

Figure 10: Desired vs actual foot position with both Joint and Cartesian PD control for a sequence of 2 seconds (optimized gains)

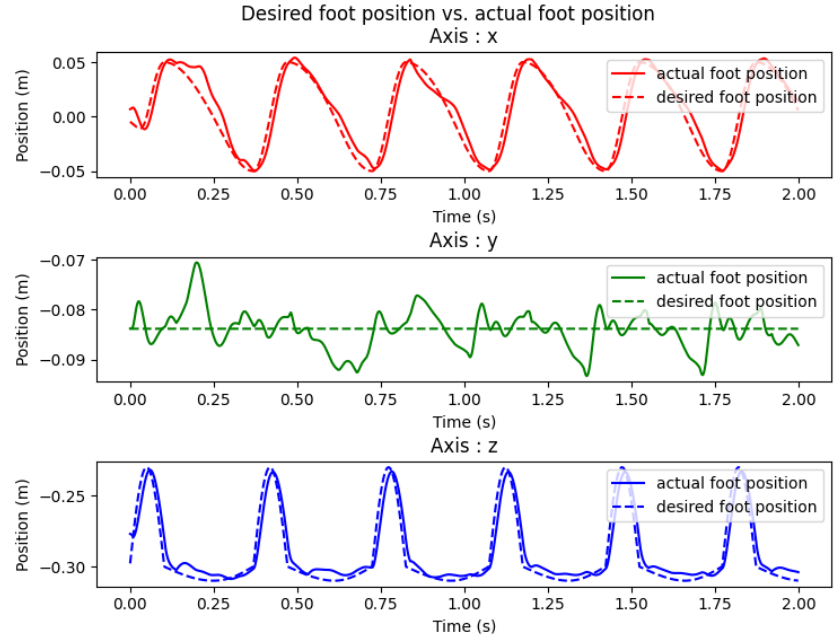

Figure 11: Desired vs actual foot position with only a Joint PD control for a sequence of 2 seconds (optimized gains)

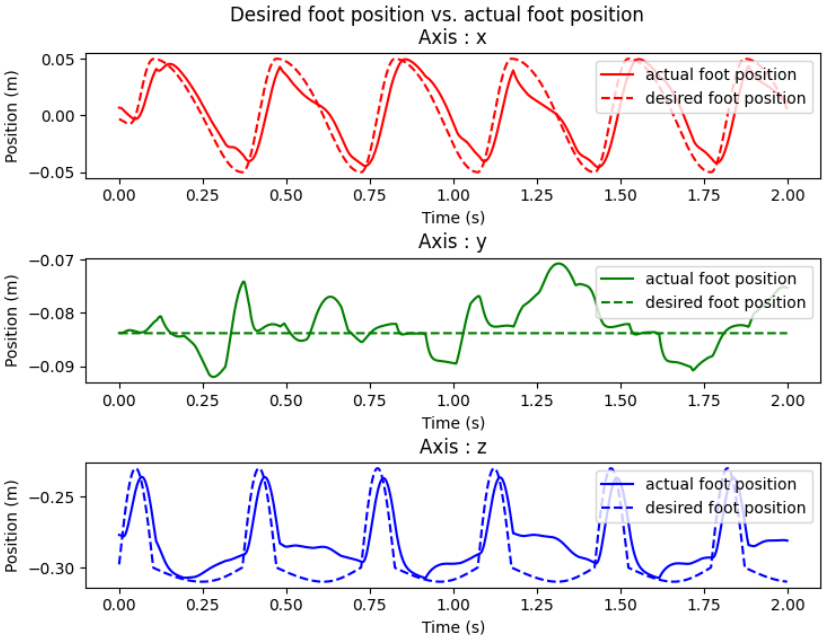

Figure 12: Desired vs actual foot position with only a Cartesian PD control for a sequence of 2 seconds (optimized gains)

In figures 13-15 we plot a comparison between the desired joint angles and the actual joint angles. We display the results for the optimized gains only. The resulting movement with the default gains is the same as previously stated in the foot position comparison. Discussion about the tracking and gains is done in section 2.3.2.

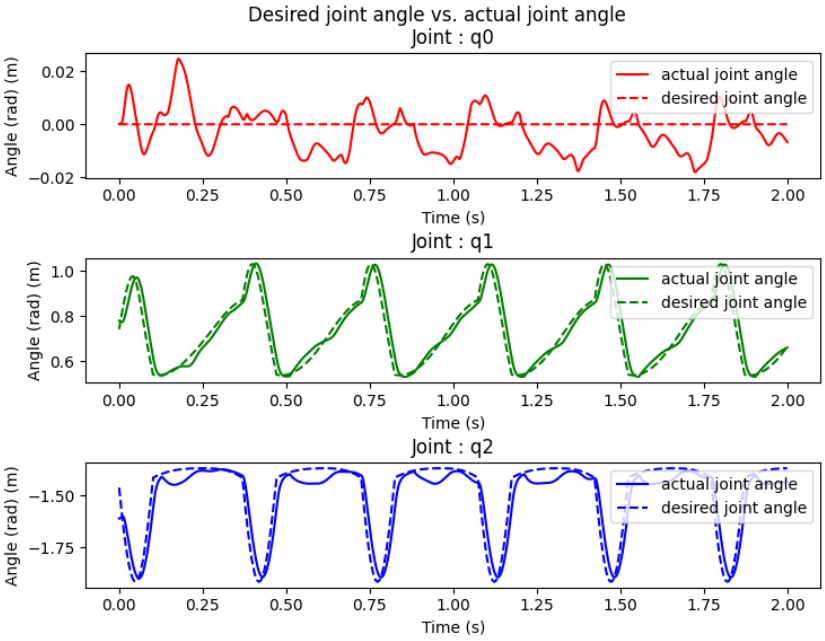

Figure 13: Desired vs actual joint angle with both Joint and Cartesian PD control for a sequence of 2 seconds (optimized gains)

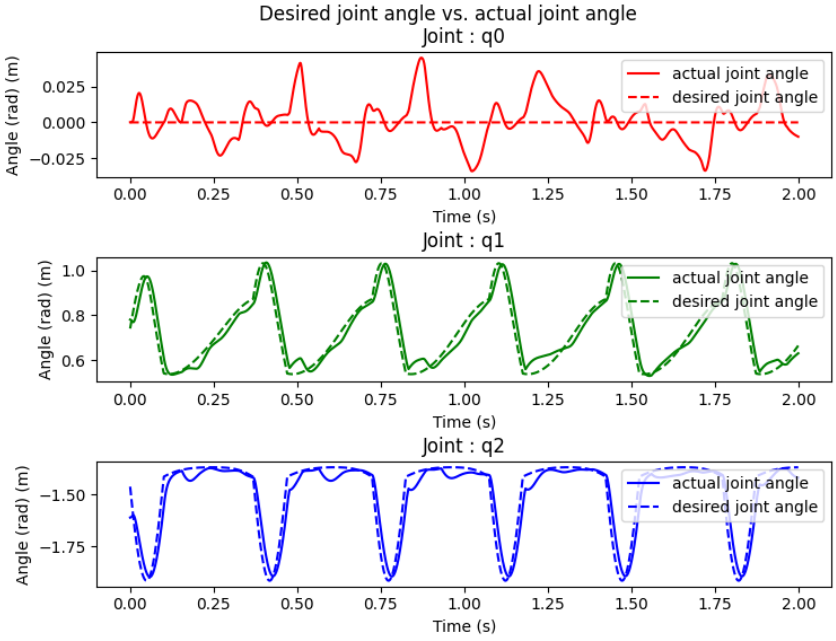

Figure 14: Desired vs actual joint angle with only Joint PD control for a sequence of 2 seconds (optimized gains)

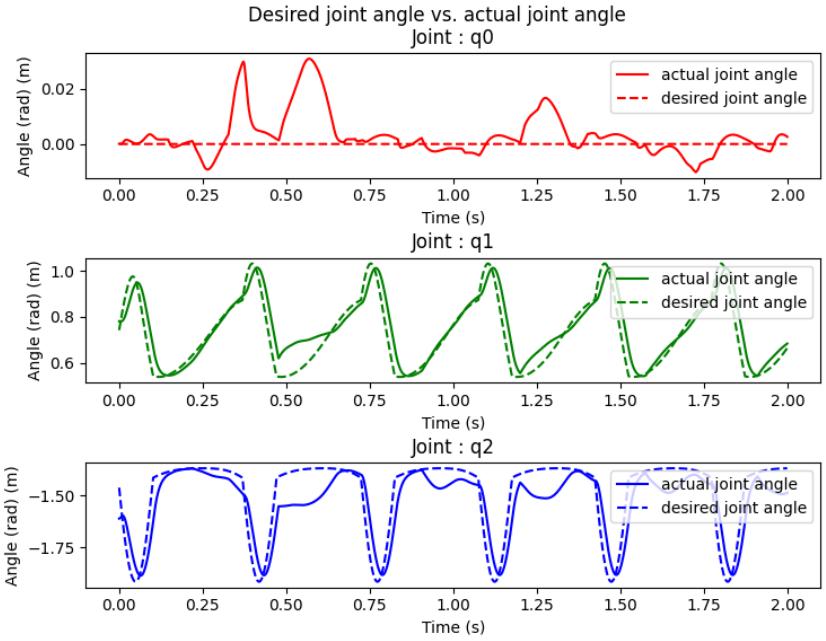

Figure 15: Desired vs actual joint angle with only Cartesian PD control for a sequence of 2 seconds (optimized gains)

2.3 Discussion

2.3.1 Desired vs actual foot position and used Gains

In figures 7-9 we have a comparison of the actual foot (front right leg) position of the robot and the desired leg position for all three cases of control (Joint and Cartesian PD, Joint PD only and Cartesian PD only) with the default gain values. Only the Joint+Cartesian and Joint PD alone works. Using Cartesian PD control alone is not enough. The gains have been tuned (Table 1) for each of these situations to overcome this problem (figures 10-12)

2.3.2 Desired vs actual joint angles and used Gains

The results of the achieved joint angles can be seen in figures 13-15. The same gains as in table 1 are used. Like in section 2.3.1, the gains have been optimized for a trot gait and achieve the best performance for a Joint + Cartesian PD controller.

PD control	$K_{p,joint}$	$K_{d,joint}$	$K_{p,cartesian}$	$K_{d,cartesian}$
Joint + Cartesian PD	100	1	1500	20
Joint PD only	200	2	-	-
Cartesian PD only	-	-	5000	80

Table 1: Tuned PD gains for the Trot gait

Adding a Cartesian PD control on top of a Joint PD controller improves its performance and adds redundancy to the control which makes it requires lower gains. When using a Cartesian PD only or a joint PD only, we lose that additional stiffness. In that case, we need to increase the gains as seen in Table 1. However, increasing the gains too much makes the joints too stiff and the movement loses its 'fluidity'. The robot also loses its ability to react to small collisions because of the non responsiveness of the joint. The required torques might also be higher that the limit torque values of the joints which can damage the robot. In general, using a moderately high torque value with multiple controls (Joint and Cartesian) is preferred. The tracking performance resulting from both Joint and Cartesian PD is the best as seen in figure 10 compared to figure 12 and 11.

2.3.3 Duty cycle, Cost of Transport and hyperparameters tuning for fast and slow gaits

In order to achieve fast locomotion, each gait was tuned individually. The parameters that were the most important for achieving that are ω_{swing} and ω_{stance} which dictate the swing and stance frequency of the movement. The control gains were also tuned but remain relatively close to the values in table 1, and similar velocities could be achieved without fine tuning them further more. The table 2 contains the resulting speed, CoT and duty cycle. In table 3 we have the same characteristics for slow gaits, where the default swing and stance frequencies were applied ($\omega_{swing} = 10\pi$, $\omega_{Stance} = 4\pi$).
In order to calculate the Cost of Transport (CoT) we use the formula:

$$CoT = \frac{P_{avg}}{mgv_{avg}} = \frac{\tau v}{mgv_{avg}} \tag{8}$$

where P_{avg} is the average power consumed over all the steps, v_{avg} the average body velocity of the robot, m the mass of the robot (equal to 12.45 kg by summing the weight of all links) and g the standard gravity acceleration (9,81 m/s^2). The power consumed per step can also be calculated by the product of the motor torques and velocities.

Gait	v_{avg} (m/s)	T_{step} (s)	T_{stance} (s)	T_{swing} (s)	D	CoT
Trot	0.612	0.272	0.197	0.073	2.64	0.543
Walk	1.434	0,098	0,049	0,023	1.67	0.756
Bound	0.626	0.331	0.267	0.062	4.24	0.842
Pace	1,376	0,119	0.059	0.059	1	0,459

Table 2: Gait characteristics for high velocities setup

Gait	v_{avg} (m/s)	T_{step} (s)	T_{stance} (s)	T_{swing} (s)	D	CoT
Trot	0.345	0.349	0.243	0.105	2.313	0.345
Walk	0.278	0,348	0,242	0,105	2.311	0.412
Bound	0.345	0.349	0.243	0.104	2.313	0.803
Pace	0.341	0,347	0.242	0.104	2,315	0,563

Table 3: Gait characteristics for low velocities setup

From the table 2, we can see that, with tuned parameters, the highest achieved average body velocity is $v_{avg} = 1.434m/s$ with the Lateral sequence walk gait, followed by the Pace gait with an average velocity of $v_{avg} = 1.376m/s$. The slowest gait is the Trot gait with an average velocity of $v_{avg} = 0.612m/s$. The two fastest gates have a relatively low duty cycle with $D = 1.67 and D = 1$ respectively while the other gates have a high duty cycle with the Bound gait running with $D = 4.24$. The Pace gait achieves the lowest Cost of Transport with $CoT = 0,459$ while still having a high velocity. When using the default swing and stance frequencies (slow movement) we can see that while all the gates have the same duty cycle, the Bounding gait still has a very high Cost of Transport and is thus not energy efficient in general.

2.3.4 Discuss how the controllers could potentially be extended with different feedback loops and with descending control signals for modulating locomotion

Much like with spinal cord system of quadruped animals which relies on sensory feedback from the environment, we could potentially extend our control system with feedback loops. Pressure sensors could be added to the system in order to generate a reflexive movements (such as flexing the knee when hitting an obstacle, or adding a stumbling movement after a hard perturbation in order to recover without falling). These feedback loop could help the robot navigate in unstable terrain, and have been demonstrated[2] to help synchronize the oscillators and generate rhythm. The system could also be further extended with model based controllers such as Virtual model control[3] which augment the CPG with control torques that are responsible for maintaining the posture of the quadruped.

3 Deep Reinforcement Learning

In this section we present the second method used for locomotion which relies on Deep Reinforcement Learning.

3.1 Method

Deep Reinforcement Learning is used in order to learn a control policy for the quadruped robot. The problem is formulated as a Markov Decision Process, which, provided with an observations space s_t, provides an action a_t from the action space and assigns a reward to that transition depending on a reward function. The policy network consists of a Two-layered perceptron, each of size 256 neurons. The policy is updated using Proximal Policy Optimization[4] (PPO) or a Soft Actor-critic algorithm[5].

3.1.1 Action space

The action space that is used to train the best policy is a CPG-RL action space where we learn to modulate CPG parameters (amplitude and phase) using reinforcement learning. The CPG variables are then mapped to the Cartesian foot xz positions. Then inverse kinematics are used to calculate the corresponding joint angles which are tracked using a Joint PD control.

3.1.2 Observation space

The observation space used to train the policy contains the default elements:

- θ_m : The motor angles of all four legs.
- $\vec{v}_m$: The velocity of the motors of all four legs.
- $\vec{q}_{base}$: The orientation of the base in quaternion format.

To which was added the following elements:

- $(v_x, v_y, v_z)_b$: The linear velocity of the base.
- $(\omega_x, \omega_y, \omega_z)_b$: The angular velocity of the base.
- r: The amplitude of the CPG.
- θ: The phase of the CPG.

When using an action space other than the CPG-RL, the CPG terms (r and θ) are removed without any other change to the observation space.

The table 4 contains the applied limits on the observation space
discussion on the added elements can be found in section 3.3.2

Observation space	Upper value	Lower value
Motor angle	UPPER_ANGLE_JOINT	LOWER_ANGLE_JOINT
Motor velocities	VELOCITY_LIMITS	- VELOCITY_LIMITS
Base orientation	1	-1
Base linear velocity	10 m/s	-10 m/s
Base angular velocity	2 rad/s	-2 rad/s
CPG amplitude	1	0
CPG phase	2π	0

Table 4: Upper and lower limits of the observation space

3.1.3 Reward function definition

In order to guide the learning of the robot, the reward function was crafted following the table 5. It consists of the terms of the default reward to which were added other stabilizing terms. Some of the terms are inspired from the lecture course[1].

Variable	Definition	weight
vel_tracking_reward	$e^{(v_{x,b}-v_{des})^2}$	0.05
yaw_reward	$\lvert\psi_b\rvert$	-0.02
drift_reward	y_b	-0.01
energy_reward	$\lvert\tau v dt\rvert$	-0.01
orientation_reward	$\lVert q_{base} - [0,0,0,1]\rVert$	-0.1
lin_vel_reward	$v_{z,b}^2$	-0.1
ang_vel_reward	$\omega_{z,b}^2$	-0.05
joint_motion_reward	$\lVert\vec{v_m}\rVert^2$	-0.001
roll_penalty	$\lvert\phi_b\rvert$	-0.2
joint_torque_penalty	$\lVert\tau\rVert^2$	-0.00002

Table 5: Reward terms and weights

3.1.4 Deep reinforcement learning definition

We trained our best policy with a PPO algorithm which we found was quicker than the SAC one. We tried changing what we thought were the most important hyper-parameters of the PPO algorithm, mainly the batch size, the number of epochs and the learning rate but we found that the default parameters are sufficient. Here is an explanation of the hyper-parameters provided (explanations mainly taken from : [6]):

- gamma : used as trade-off to chose if a reward will be optimized in short term or long term. The closer it is to 1 the more importance we give to the future rewards.
- n_steps : It is the number of steps to run for each environment per update.
- ent_coef : The entropy coefficient is a measure of how much "risk" the policy will take when training. Meaning that if a big entropy coefficient is set, the policy will tend to explore more actions.
- learning_rate : It is a measure of the "steps" of the learning of a policy. If the learning rate is too big the policy will never converge and could even diverge. If it is set too low, the learning will be very slow.
- vf_coef : The value function is a function which represent the long term value of being in a certain state, it helps the agent to make a decision in what action to do next. The value function coefficient is the "weight" given to that function.
- max_grad_norm : During training the gradient can take huge values, this hyperparameter is used to clip the gradient at a given value.
- gae_lambda : The generalized advantage estimator estimates the advantage function which is a measure of how much better an action is compared to the average action value. It is used to help an agent choose between two actions that both lead to high rewards. The gae_lambda is a "weight" given to this generalized advantage estimator.

- batch_size : It is the number of samples used to update the learning algorithm. A large batch size leads to more stable and accurate gradients but could results in very slow learning.
- n_epochs : It represents the number of times the algorithm will go over the whole dataset. Larger epochs lead to better trained agent but also to slower training.
- clip_range : It used to control the magnitude of the policy update to stays within the bounds of a clipped region around the old policy.
- clip_range_vf : Similarly to the last hyperparameter, the clip_range_vf specifies a range around the old value function in which the new one has lie.
- verbose : It is simply the amount of information printed during the training.
- tensorboard_log : It controls whether or not the algorithm logs information to tensorboard.
- _init_setup_model : It simply states wether or not to build the network at the creation of the instance.
- policy_kwargs : It allows to pass additionnal arguments to the policy.
- device : It states whether the code should be run on cuda or cpu.

3.1.5 Environment changes

When training, we only used the basic environnement but then we did test our policy with different environment changes. The first tests we did was with the competition environment which has strips of colors representing the different friction coefficient. On this environment we could see that our best policy almost reached the end of the "arena". We found that what our policies had the most troubles with was the difference in height of certain platforms that "dissyncronized" their movement and led to either the robot dropping or tedious movements.

3.2 Results

In this section, all the results are shown and then discussed in the Discussion section along with all the other questions that were raised.

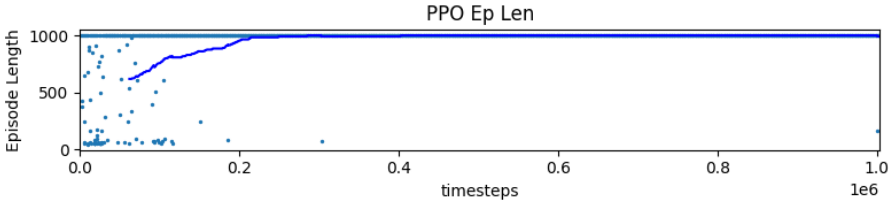

Figure 16: Episode length progression for the best policy using CPG-RL

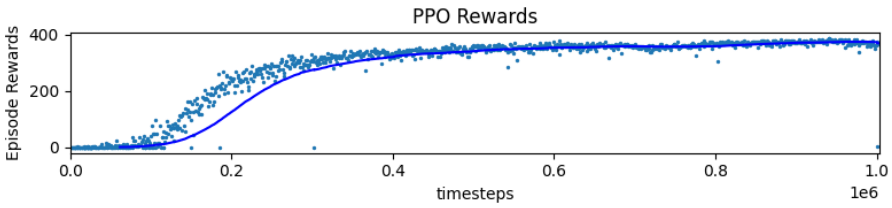

Figure 17: Reward progression for the best policy using CPG-RL

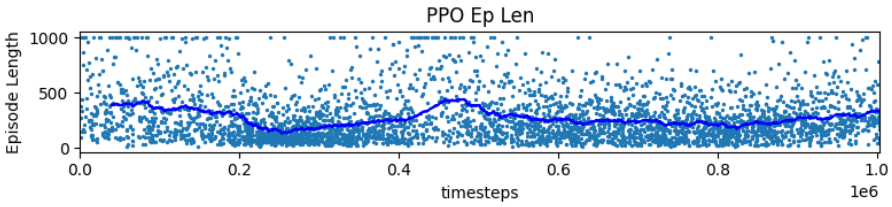

Figure 18: Episode length progression for a policy using PD action space with the same setup of the best policy (CPG-RL)

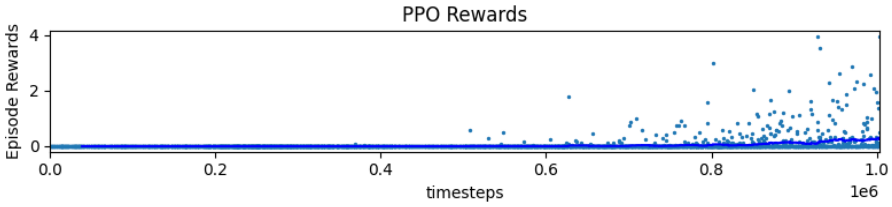

Figure 19: Reward progression for a policy using PD action space with the same setup of the best policy (CPG-RL)

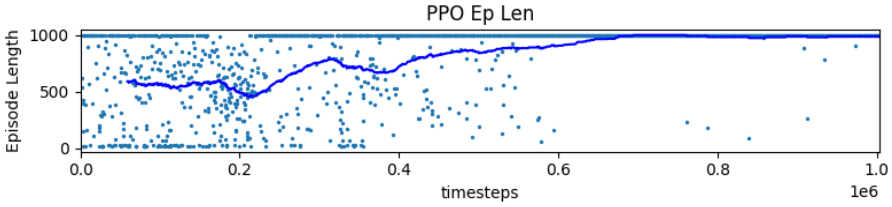

Figure 20: Episode length progression for a policy using Cartesian PD action space with the same setup of the best policy (CPG-RL)

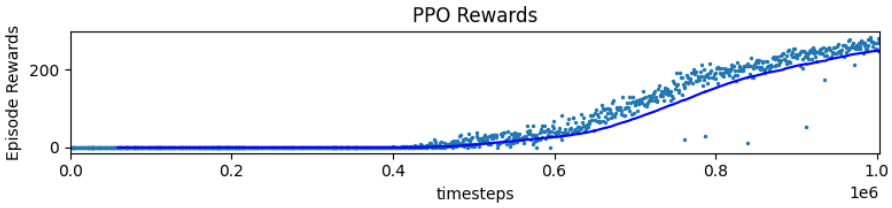

Figure 21: Reward progression for a policy using Cartesian PD action space with the same setup of the best policy (CPG-RL)

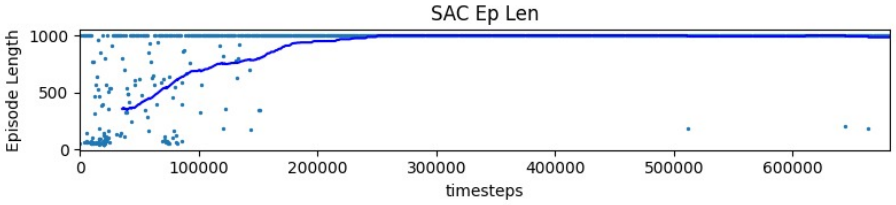

Figure 22: Episode length progression for a policy using CPG-RL action space trained with SAC

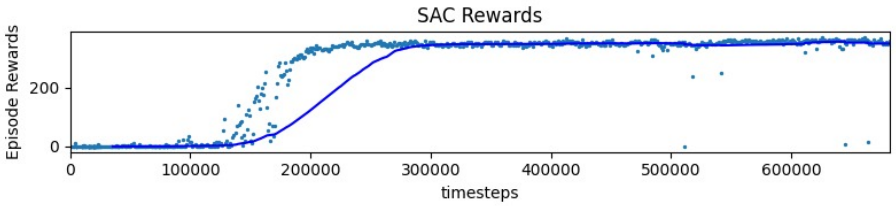

Figure 23: Reward progression for a policy using CPG-RL action space trained with SAC

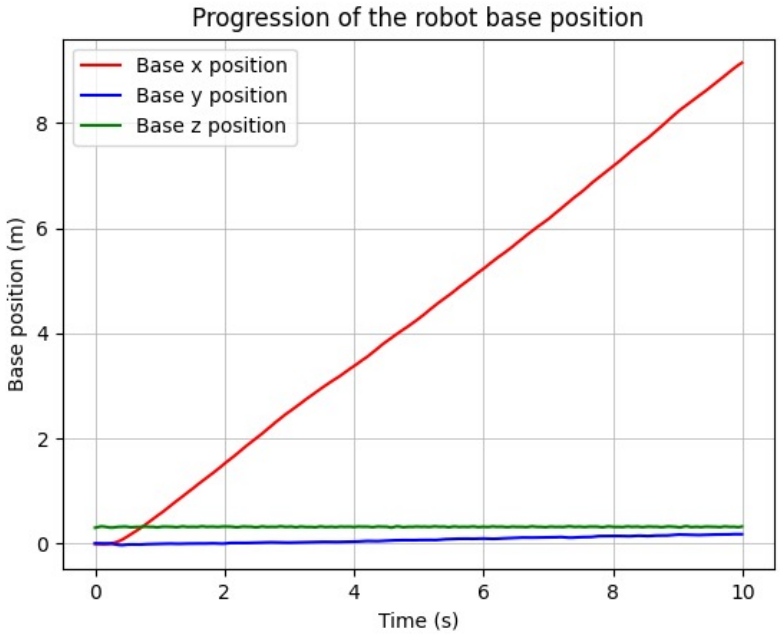

Figure 24: Progression of the base position using the best CPG-RL policy

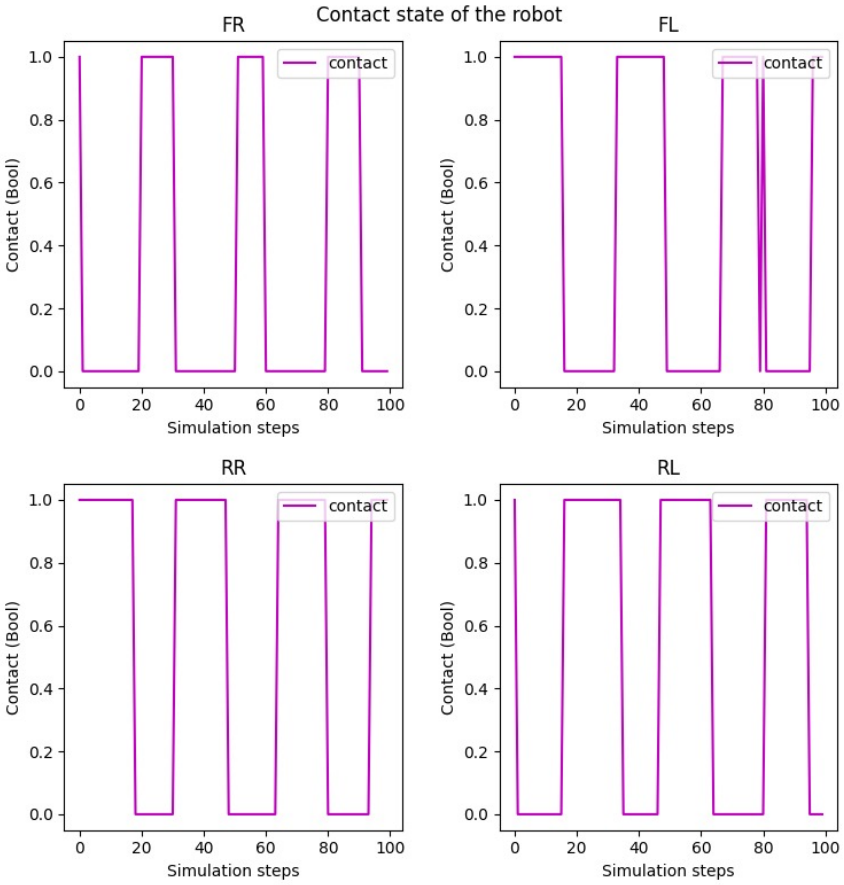

Figure 25: Leg contact boolean using a CPG-RL policy

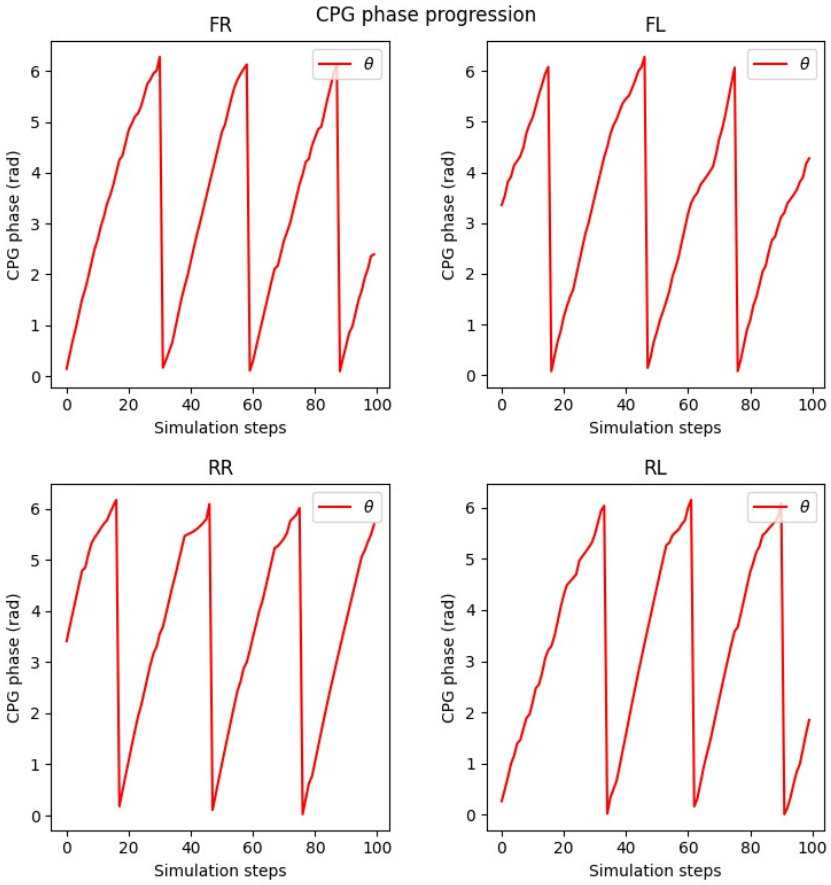

Figure 26: CPG phase progression of the robot for a CPG-RL policy

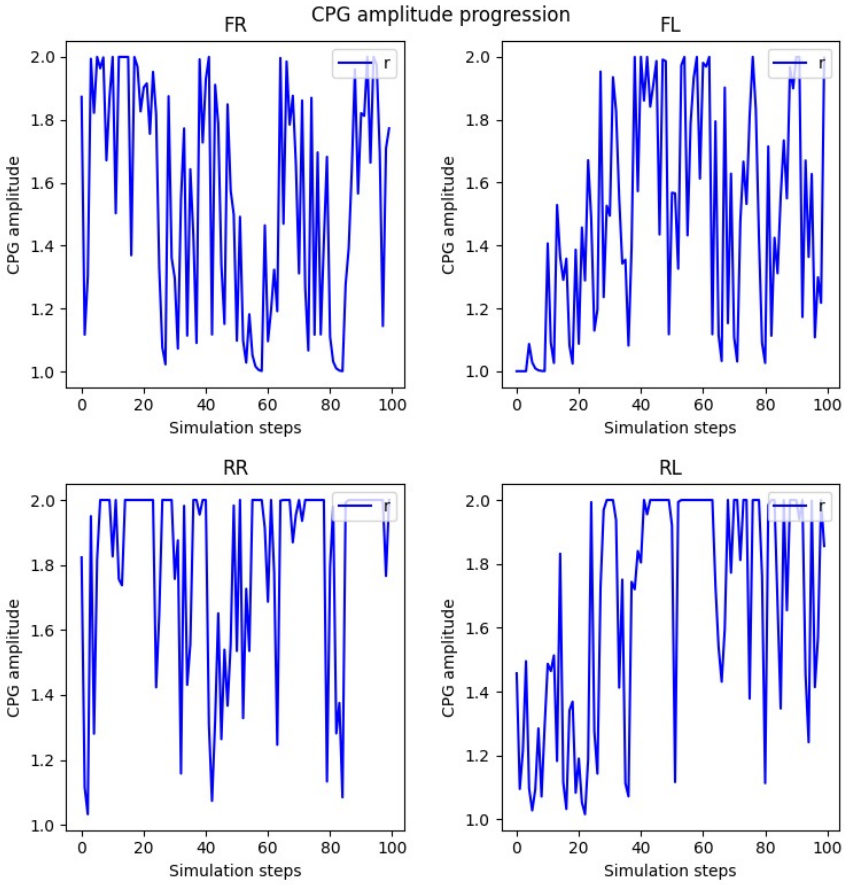

Figure 27: CPG amplitude progression of the robot for a CPG-RL policy

3.3 Discussion

In this section we will discuss some of the main questions about the reinforcement learning part which will follow the same structure as the Method section.

3.3.1 Action space comparison

The CPG-RL action space holds the best performance in term of rewards and episode length as seen in figures 17 and 16. The episode length converges to its optimal value (1000) after around two hundred thousands steps, while the reward converges after around four hundred thousand steps. The variance of the reward is also relatively low. When using the same training conditions with other action spaces, the training does not produce good results. Figures 18 and 19 show the results of training the policy with a regular PD as action space. The episode length does not converge and the reward stagnate around zero. A policy trained with a Cartesian PD action space performs better than the regular PD as seen in figures 20 and 21 but produces a failing policy as seen in the simulation video attached to this paper. The reward function and observation space have

been tuned for the CPG action space which explains why using other actions spaces fails. Overall, the CPG-RL produces the highest values in term of reward and converges the fastest in episode length. It is followed by a Cartesian PD controller that converges but produces a faulty policy and finally the regular PD controller that doesn't converge.

3.3.2 Observation space discussion

We saw in the Observation space section how the observation space is set up. Here we discuss the main reason for choosing each measurement:

- Motor angles : Providing the network with the motor angle seemed a logical way to constrain our motor and force the policy into taking actions that respect the maximum allowed angles.
- Motor velocities : Measures of the motor velocities were also added for similar reasons as the angles. This term is more important in cases such as training highly dynamic gaits. This term makes sure that the final policy respects the torques than could be generated.
- Base orientation : Including the base orientation is a important part of navigating the robot. We expected that the trained policy would learn to avoid observation spaces with irregular orientations.
- Base linear velocity : We expect this term to help the robot recognize the states with a desired velocity. As we penalize the robot for lateral drifts and reward it for tracking the forward velocity, this term seemed important to help the robot track a given velocity with a steady pace
- Base angular velocity : This observation results from the same thought of process as the previous one. Most of the failing policies are due to undesirable angular movements (either facing, facing down or sideways). The angular velocity state is added in conjunction with orientation penalties in the reward function in order to reduce the oscillations in those undesired directions.
- CPG amplitude : Since we used CPGs to control our robot, it was reasonable to include this basic CPG state. This could help the robot converge to s stable CPG amplitude.
- CPG phase : Much like the aplitude, the phase state was added in order to give the robot a notion of the phase which could help stabilize the movement.

Overall, many observation terms were tested and only the most relevant terms were used. The biggest performance improvement was observed after adding the base linear velocity which helped stabilize most of the policies and produce somehow "smooth" movement.
If we think about how these measurements would be in real world, the joints angles and velocities errors could be manageable if the robot includes specific proprioceptors working with a very high frequency. CPG amplitude and phase rely on theroretical controller and are much harder to quantify, the measurements could be very noisy. The base orientations could be potentially measured

precisely with a good IMU. Concerning the base position, since the measurement are not directly on the actuators it has to be done either with calculations based on the limb positions and velocities or with an external sensors (possibly with computer vision) which induces a relatively high error and possibly noisy measures.

3.3.3 Reward function discussion

We already described what terms are included in our reward function (5), in this section we will go more into detail on the choice of different components :

- Velocity tracking : A major (default) term which pushes the robot to track a desired forward velocity.
- Yaw penalty : This term reduces lateral movements and penalizes the robot for not following a straight line.
- Drifting penalty : Same as the previous term, we want our robot to go in a straight line, after compensating for the yaw we want also to compensate for the drift. Combining this term with the previous one, we force the policy to go in a straight line.
- Energy penalty : The purpose of this whole training is to train a policy which could be implemented in real world. Minimizing the consumed energy is then important and helps in producing fluid movements.
- Orientation penalty : This term forces the robot to adopt neutral orientations, much like the yaw and drifting penalty, this term produces redundancy to the system.
- Height velocity penalty (lin_vel_reward) : This term was a major addition to the reward function. It penalizes the robot for oscillating in high. Overall we observed that the produced policies have very unsteady robot heights, and adding this terms corrects this issue and also helps the robot adopt a horizontally stable state.
- Angular velocity penalty : When running several policies without this component we noticed that the robot would rear like a horse. To prevent that and to have a steady body we included this term.
- Joint motion penalty : Following the same thought of process as for the energy component we followed what was included in the course [1] which minimizes the speed of the limbs to have a more natural movement. This component is a bit redundant with the energy term but taking them both produces good results.
- Roll penalty: A term added in conjuction with angular velocity reward to limit both the movement and oscillations in the roll direction.
- Joint torque penalty : Following the trend of minimizing the energy, we want our robot to be able to transfer in a real world application as easy as possible. We know that motors can't sustain infinite amount of torque leading us to implement a torque penalty component to prevent the policy from discovering states with unreasonable torques.

To choose the weight we followed this principle :

1. We select the term that we want to tune and cancel all the other terms in the reward function
2. We train the policy with the desired term only in order to quantify the magnitude of the term.
3. We repeat the first two steps for all the terms.
4. We divide each term by its magnitude (or a relatively close value) to have "almost" standarized terms.
5. We further fine tune the weight to give slight advantage to terms that we deem more important than other.
6. We test and confirm that our policy works well

3.3.4 Deep reinforcement learning algorithm discussion

As stated in section 3.1.4, PPO algorithm was preferred over the SAC algorithm. In figures 16, 17 we have the results of the best trained policy using PPO algorithm, while in figures 22, 23 we have the results of training the same policy with a SAC algorithm. We notice that SAC takes much more time to train, in fact the average episode length converges much later than when using PPO. PPO also achieves overall a higher final reward than SAC. The two algorithms are in principle very different. While PPO uses an on-policy approach, meaning that it learns its value function from observations made by the current policy exploring the environment, SAC uses off-policy learning which means that it can use observations made by previous policies' exploration of the environment. On-policy algorithms tend to be more stable but data hungry, whereas off-policy algorithms tend to be the opposite.

When it comes to the exploration, exportation tradeoff, The PPO algorithm encourages exploration by using entropy regularization, which prevents agents from converging to local optima. The SAC algorithm strikes an exceptional balance between exploration and exploitation by adding entropy to its maximization objective. In general, SAC algorithms are a bit harder to tune than PPO algorithms but produce equally good results. The deciding factor for training our policy was ultimately the shorter training time for the PPO algorithm

3.3.5 Duty cycle, Cost of Transport and average speed

In table 6 are summarized the gait characteristics of the fastest policy and a regular slow policy. Both policies have a CPR-RL action space as mentioned before and both are trained using PPO algorithm.

Gait	v_{avg} (m/s)	T_{step} (s)	T_{stance} (s)	T_{swing} (s)	D	CoT
Fast gait	0.944	0.28	0.150	0.129	1.16	0.300
Slow gait	0.491	0,389	0,234	0,154	1.51	0.447

Table 6: Gait characteristics results for RL trained policies

From the table 6, we can see that, for the fast gait, the achieved average body velocity is $v_{avg} = 0,944m/s$ while the desired velocity was of $v_{avg} = 1m/s$.The slow gait achieves a velocity of $v_{avg} = 0.491m/s$. while the actual desired velocity is $v_{avg} = 1m/s$. Both gates have a relatively low duty cycle with $D = 1.16 and D = 1,51$ meaning that the quadruped is almost alternating symmetrically between swing and stance phases. The fast gait achieves the lowest Cost of Transport with $CoT = 0,300$, while the slow gait has a $CoT = 0,300$. Figures 24, 25,27 and 26 contain the most relevant states for the fast policy mentionned in table 6.
We could also potentially train a policy that we can command at test time by using a continuous action space that includes a full ranges of speeds and use a specific controller (such as a high level hierarchical policy)that maps the desired speed to the optimal actions. This method is very time consuming but would allow modulating the speed as desired within a certain range.
Uneaven terrains are however very hard to deal with our current policy. A fast solution could be using vision based sensors. That translates in the simulation to adding a heigh map around the robot so that the policy could learn to react to different terrains. Training the policy with uneaven terrains remains however a much more robust solution. Augmenting the policy with a planning methods[7] could also be possible, the policy would then learn where to place the robot's feet given a map of the environment.

3.3.6 Sim-to-real transfer

We think that our approach is moderately robust and that a transfer to real world could potentially fail. We believe that the behaviors developed by our policies are heavily dependant on the simulator, as such, when transferred to a real robot, the control could fail to respect the constraints of that particular robot which results in the robots crashing directly. Furthermore, due to modelling errors (from the CPG oscillators i.e.), strategies that are successful in simulation may not transfer well. When transferring to hardware, the noise present in the sensors is very different from simulation, even after artificially adding noise to the simulation, it is very difficult to quantify the noise when running a robot in real conditions and that could ultimately play a major role in the tuning of the parameters. Another aspect to take into account in the transfer speed of the data between different modules. The many channels bandwidths could be a limiting factor not taken into account in simulation. Many methods could however be used to help the sim-to-real transfer such as Domain Optimization algorithms [8] or using Deep Reinforcement Learning to learn how to model the noise [9].

4 Conclusion

In conclusion, we went through two main methods used for locomotion, the first was the CPG, where we saw how to model a CPG oscillator and the effects of modulating its hyperparameters such as swing and stance phases to achieve various speeds. We also learned how to modulate the coupling matrices to achieve different gates. The second was the reinforcement learning method where we trained multiple policies with different optimization algorithms and

reward functions to achieve the desired speeds. We also saw the importance of designing the observation space, the action space, and the reward functions. For both parts we also discussed different control method which are joint control and Cartesian control and we discussed the cost of transport and duty-cycles.

Both methods are very interesting in their own way. The CPG method is very easy to implement and offers high versatility with its ability to change gait with a simple phase shift. The reinforcement learning method is more time consuming and requires more knowledge in terms of reward function crafting, but offers general policies that could work in different scenarios.

The implementation in real world applications is missing in both cases, future works could include tackling the sim-to-real transfer and testing the proposed solutions to overcome this problem.

References

[1] Auke Ijspeert, Guillaume Bellegarda, "Legged locomotion: Trajectory optimization, machine learning, and bio inspired control," @EPFL 2022.

[2] S. Suzuki, T. Kano, A. J. Ijspeert, and A. Ishiguro, "Sprawling quadruped robot driven by decentralized control with cross-coupled sensory feedback between legs and trunk," *Frontiers in Neurorobotics*, vol. 14, 2021.

[3] M. Ajallooeian, S. Pouya, A. Sproewitz, and A. J. Ijspeert, "Central pattern generators augmented with virtual model control for quadruped rough terrain locomotion," in *2013 IEEE International Conference on Robotics and Automation*, pp. 3321–3328, 2013.

[4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *CoRR*, vol. abs/1707.06347, 2017.

[5] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," *CoRR*, vol. abs/1801.01290, 2018.

[6] S. Baseline, "Ppo." `https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html`, 2022 (accessed january 6, 2022).

[7] M. Kalakrishnan, J. Buchli, P. Pastor, M. Mistry, and S. Schaal, "Learning, planning, and control for quadruped locomotion over challenging terrain," *I. J. Robotic Res.*, vol. 30, pp. 236–258, 02 2011.

[8] K. A. Hamed, Wen-Loong, and A. D. Ames, "Dynamically stable 3d quadrupedal walking with multi-domain hybrid system models and virtual constraint controllers," 2018.

[9] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," *CoRR*, vol. abs/2109.11978, 2021.